

Diagramma delle Classi - Costruzione

Trentin Tommaso

December 27, 2025

Contents

1	Livelli di dettaglio	2
2	Astrazione del System Domain Model	2
2.1	Identificare Classi	2
2.1.1	Analisi dei nomi	2
2.2	Identificare Attributi	2
2.3	Identificare Associazioni	2
2.4	Identificare Generalizzazioni e Interfacce	3
2.5	Identificare Responsabilità	3
2.5.1	Class Responsibility Collaboration (CRC) Cards	3
3	System Domain Model	4
3.1	Esempio	4

1 Livelli di dettaglio

Diagrammi UML possono essere creati in diverse fasi del ciclo di vita, con diversi scopi / livelli di dettaglio:

- **System domain model:** modella entità del dominio del problema che saranno presenti nel sistema (dati di interesse), può contenere le responsabilità delle classi (opzionale);
- **System model:** include anche classi che saranno usate per costruire il sistema completo (classi per interfaccia utente, classi associate a parti di architettura, classi di utility), oltre a metodi e informazioni sulla navigabilità.

2 Astrazione del System Domain Model

1. Identificare primo insieme di **classi** candidate;
2. Aggiungere **associazioni** ed **attributi** delle classi;
3. Trovare **generalizzazioni**;
4. Trovare principali **responsabilità** di ogni classe;
5. Iterare processo finché modello ottenuto è soddisfacente.

2.1 Identificare Classi

Classi dovrebbero corrispondere a entità del dominio del problema, si applicano i concetti di OO: una classe identifica un tipo di dato astratto. Una buona classe:

- ha un nome che ne rispecchia l'intento;
- è un'astrazione che modella un elemento del dominio del problema;
- ha insieme ridotto e ben definito di responsabilità.

2.1.1 Analisi dei nomi

Semplice tecnica per scoprire le classi del dominio:

1. Analizzare documentazione di partenza;
2. Elencare i nomi;
3. Eliminare nomi che:
 - sono ridondanti;
 - rappresentano / caratterizzano istanze e non classi;
 - sono vaghi / generici;
 - corrispondono a classi non necessarie al livello considerato.

2.2 Identificare Attributi

- Cercare informazioni da conservare per ciascuna classe;
- Possibile che nomi scartati come classi al passo 1 ora possano essere considerati attributi;
- Una classe non dovrebbe avere troppi attributi;
- Bisogna stare attenti agli attributi contenenti molteplici valori.

2.3 Identificare Associazioni

- Espressioni verbali che coinvolgono più classi indicano possibili relazioni tra esse;
- Vanno cercate nell'ordine:
 1. **Gen-Spec:** espressioni del tipo "è un";
 2. **Contenimento:** espressioni del tipo "è fatto di", "comprende";
 3. **Associazione:** ogni altra espressione.
- Una volta identificata un'associazione:
 - specificare molteplicità da entrambi i lati;

- assegnare nome chiaro all'associazione e / o definire ruoli delle entità che partecipano all'associazione;

2.4 Identificare Generalizzazioni e Interfacce

Due modi principali per trovare **generalizzazioni**:

- Bottom-Up - raggruppando classi simili creando una nuova super-classe;
- Top-Down - creando prima classi più generali e poi specializzando.

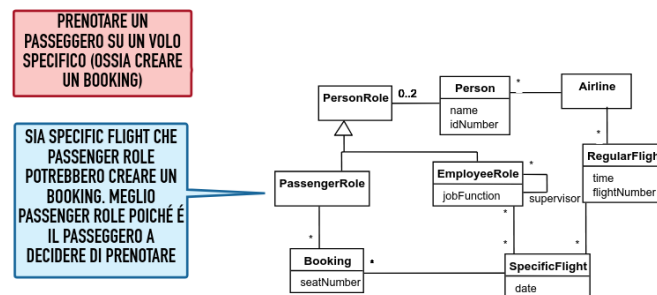
Si crea invece un'**interfaccia** se:

- classi sono molto diverse fra loro, tranne per alcune operazioni in comune;
- potrebbero essere disponibili diverse implementazioni della stessa classe.

2.5 Identificare Responsabilità

- **Responsabilità** corrisponde a una funzionalità richiesta al sistema SW;
- Responsabilità di ogni requisito funzionale va attribuita ad una delle classi, anche se nella pratica potrà essere svolto mediante collaborazione fra più classi;
- Se classe ha troppe responsabilità, valutare di dividerla in più classi;
- Se classe non ha responsabilità, probabilmente è inutile;
- Se una responsabilità non può essere attribuita ad alcuna delle classi esistenti, probabilmente ne va creata una nuova;
- Responsabilità di creare istanze di una classe non può essere attribuita alla classe stessa, ma ad una classe ad essa collegata (stessa cosa per collections tipo lista, catalogo... di);

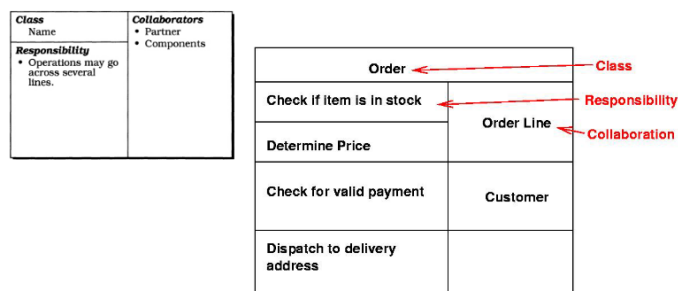
ESEMPIO: COMPAGNIA AEREA CON RESPONSABILITÀ



2.5.1 Class Responsibility Collaboration (CRC) Cards

Metodo alternativo per ottenere diagramma delle classi:

1. Per ogni classe identificata si pone nome della classe su una scheda (Card);
2. Quando vengono individuati attributi / responsabilità, si elencano sulle Card;
3. Si sistemano le card su una board per creare il Class diagram;
4. Si disegnano linee corrispondenti ad associazioni e generalizzazioni.



3 System Domain Model

Nel progetto di dettaglio bisogna passare dalle responsabilità alle operazioni necessarie a realizzare le responsabilità di ciascuna classe: ognuna di queste può essere realizzata attraverso molteplici operazioni. Le operazioni che implementano una responsabilità sono normalmente dichiarate *public*, mentre altri metodi che collaborano a realizzarle devono essere dichiarate, se possibile, *private*.

3.1 Esempio

